



King Abdul-Aziz University
Faculty of Computing and Information Technology
Computer Science Department
Algorithms and Data Structures (II)
CPCS-324 Spring 2023 Group Project – Part I



EMPIRICAL ANALYSIS OF KRAUSKAL AND PRIM ALGORITHMS

Project Team:

Name	ID	Section
Razan Arif Alamri		
Shatha Kalid Binmahfouz		
Khlood Homiid Alsofyani		
Leen Baqalaql		



Table of Contents:

1. Introduction	4
2. Kruskal Algorithm:	4
3. Prim's using min-heap Algorithm:	5
4. Empirical Analysis.....	6
5. Difficulties Faced During the Phase Design	10
6. Conclusion	10
7. References	11
8. Appendix	11

Illustrations:

TABLE 1: KRUSKAL'S RUNTIMES.....	7
TABLE 2: PRIM RUNTIMES	7
FIGURE 1: KRUSKAL ALGORITHM	4
FIGURE 2: PRIM'S USING MIN-HEAP ALGORITHM	5
FIGURE 3: OUTPUT OF KRUSKAL ALGORITHM FOR GRAPH READ FROM FILE	6
FIGURE 4: OUTPUT OF PRIM ALGORITHM FOR GRAPH READ FROM FILE	6
FIGURE 5: AVERAGE RUN TIME OF KRUSKAL'S ALGORITHM.....	8
FIGURE 6: AVERAGE RUN TIME OF PRIM'S ALGORITHM	8
FIGURE 7: OUTPUT1 OF KRUSKAL AND PRIM ALGORITHMS FOR RANDOMLY GENERATED GRAPHIC..	9
FIGURE 8: OUTPUT1 OF KRUSKAL AND PRIM ALGORITHMS FOR RANDOMLY GENERATED GRAPHIC..	9

1. Introduction

In this project, we are given a problem related to phone network design where we need to connect several offices using phone lines. The objective is to find a set of lines that connects all the offices with a minimum total cost, forming a minimum spanning tree. Two algorithms, Kruskal and min-heap-based Prim, will be used to compute the minimum spanning tree of a given graph and print it. Furthermore, we will define a function to randomly generate graphs with varying sizes and edge weights and perform an experimental comparison of the Kruskal and Prim algorithms on these randomly generated graphs. The aim of this project is to analyze the efficiency of these algorithms and identify the situations in which one algorithm may perform better than the other.

2. Kruskal Algorithm:

Kruskal's algorithm is used to find the minimum spanning tree of a connected undirected graph. It works by sorting the edges of the graph in ascending order of their weights and then iteratively adding the edges to the minimum spanning tree, as long as they do not form a cycle. Kruskal's algorithm is a greedy algorithm, meaning that it makes locally optimal choices at each step with the hope of finding a global optimum.

ALGORITHM *Kruskal(G)*

//Kruskal's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph $G = (V, E)$

//Output: E_T , the set of edges composing a minimum spanning tree of G

sort E in nondecreasing order of the edge weights $w(e_{i1}) \leq \dots \leq w(e_{i|E|})$

$E_T \leftarrow \emptyset$; $ecounter \leftarrow 0$ *//initialize the set of tree edges and its size*

$k \leftarrow 0$ *//initialize the number of processed edges*

while $ecounter < |V| - 1$ **do**

$k \leftarrow k + 1$

if $E_T \cup \{e_{ik}\}$ is acyclic

$E_T \leftarrow E_T \cup \{e_{ik}\}$; $ecounter \leftarrow ecounter + 1$

return E_T

Figure 1: Kruskal Algorithm

3. Prim's using min-heap Algorithm:

Prim's algorithm also used to find the minimum spanning tree of a connected undirected graph. The algorithm works by starting at any vertex of the graph and iteratively adding the adjacent edge with the smallest weight to the minimum spanning tree. However, instead of sorting the edges like Kruskal's algorithm, Prim's algorithm uses a min-heap data structure to efficiently keep track of the smallest edge weight. The min-heap data structure allows for constant time access to the smallest weight edge, making the algorithm more efficient for dense graphs.

ALGORITHM *PrimMinHeap(G)*

//Prim's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph $G = (V, E)$

//Output: E_T , the set of edges composing a minimum spanning tree of G

Initialize (Q)

foreach $u \in V$ adjacent to v_0 **do**

 Insert (Q, u, w_u)

$V_T \leftarrow \{v_0\}$

$E_T \leftarrow \emptyset$

for $i \leftarrow 1$ to $|V| - 1$ **do**

// FIND A MINIMUM - WEIGHT EDGE $e^ = (v^*, u^*)$ AMONG ALL (v, u)*

// SUCH THAT v IS IN V_T AND u IS IN $V - V_T$

$u^* \leftarrow \text{Delete Min} (Q)$

$V_T \leftarrow V_T \cup \{u^*\}$

$E_T \leftarrow E_T \cup \{e^*\}$

foreach $u \in V - V_T$ adjacent to u^* **do**

 Insert (Q, u, w_u)

return E_T

Figure 2: Prim's using min-heap Algorithm

4. Empirical Analysis

- The Problem:

“.. a problem like phone network design. You have a business with several offices; you want to lease phone lines to connect them up with each other, and the phone company charges different amounts of money to connect different pairs of cities. You want a set of lines that connects all your offices with a minimum total cost. It should be a spanning tree, since if a network isn't a tree you can always remove some edges and save money.”

- Requirement 1:

1. Compute the minimum spanning tree and print it using two algorithms as follows:

```
-----
                        Welcome to Phone Network Design Program
                        -----

This program to Compute the minimum spanning tree using two algorithms:
1- Kruskal's Algorithm
2- Min-heap based Prim algorithm

-----

<< Test Type >>

1: Read The Graph From File
2: Create a Random Graph

> From these choices, please enter your choice: 1
-----

The phone network (minimum spanning tree) generated by Kruskal algorithm is as follows:

Office No.A - Office No.B : line length: 5
Office No.A - Office No.C : line length: 10
Office No.C - Office No.D : line length: 20
Office No.C - Office No.E : line length: 25
Office No.C - Office No.F : line length: 30

The cost of designed phone network: 18
-----
```

Figure 3: Output of Kruskal Algorithm for graph read from file

```
-----

The phone network (minimum spanning tree) generated by min-heap based Prim algorithm is as follows:

Office No.A - Office No.B : line length: 5
Office No.A - Office No.C : line length: 10
Office No.C - Office No.D : line length: 20
Office No.C - Office No.E : line length: 25
Office No.C - Office No.F : line length: 30

The cost of designed phone network: 18
-----

Min-heap based Prim Algorithm's run time is 19.0 ms.

-----

                        Thank You For Using Phone Network Design Program
                        -----
```

Figure 4: Output of Prim Algorithm for graph read from file

- Requirement 2:

2. Perform an experimental comparison of two of the minimum-spanning-tree algorithms, Kruskal and min-heap- based Prim. Develop an extensive set of experiments to test and plot the running times of these algorithms using the above randomly generated graphs.

- Kruskal's runtimes:

This table is used to calculate the total, average, best case and worst case for the data of Kruskal algorithm. X-axis represents case number with specific number of integer edges. Also, the y-axis represents corresponding values.

n	m	1	2	3	4	5	6	7	8	9	10	Total	Average	Best	Worst
1000	10000	65	44	43	58	48	46	44	42	94	60	544	54.4	94.0 ms.	44.0 ms.
1000	15000	75	47	47	48	50	52	46	60	48	49	522	52.2	75.0 ms.	46.0 ms.
1000	25000	54	50	49	49	55	61	71	50	51	51	541	54.1	71.0 ms.	49.0 ms.
5000	15000	46	44	46	46	45	44	44	48	47	44	454	45.4	48.0 ms.	44.0 ms.
5000	25000	56	56	51	51	52	51	49	50	59	62	537	53.7	62.0 ms.	49.0 ms.
10000	15000	56	48	47	51	52	46	48	50	47	46	491	49.1	56.0 ms.	46.0 ms.
10000	25000	51	52	54	62	50	53	51	51	59	53	536	53.6	62.0 ms.	50.0 ms.

Table 1: Kruskal's runtimes

- Prim's runtimes:

This table is used to calculate the total, average, best case and worst case for the data of Kruskal algorithm. X-axis represents case number with specific number of integer edges. Also, the y-axis represents corresponding values.

n	m	1	2	3	4	5	6	7	8	9	10	Total	Average	Best	Worst
1000	10000	60	44	43	36	40	44	46	43	40	46	442	44.2	60.0 ms.	36.0 ms.
1000	15000	49	47	45	47	48	49	49	52	41	48	475	47.5	52.0 ms.	41.0 ms.
1000	25000	50	56	49	52	48	49	51	49	48	54	506	50.6	56.0 ms.	48.0 ms.
5000	15000	48	39	52	50	50	51	43	38	49	40	460	46	52.0 ms.	38.0 ms.
5000	25000	57	56	51	48	48	51	49	39	50	53	502	50.2	56.0 ms.	39.0 ms.
10000	15000	48	46	48	38	55	43	48	43	48	49	466	46.6	55.0 ms.	38.0 ms.
10000	25000	48	48	44	41	48	41	42	46	59	44	461	46.1	59.0 ms.	41.0 ms.

Table 2: Prim runtimes

- The runtime average for each run's ten trials is shown in the plot for Kruskal's algorithm:

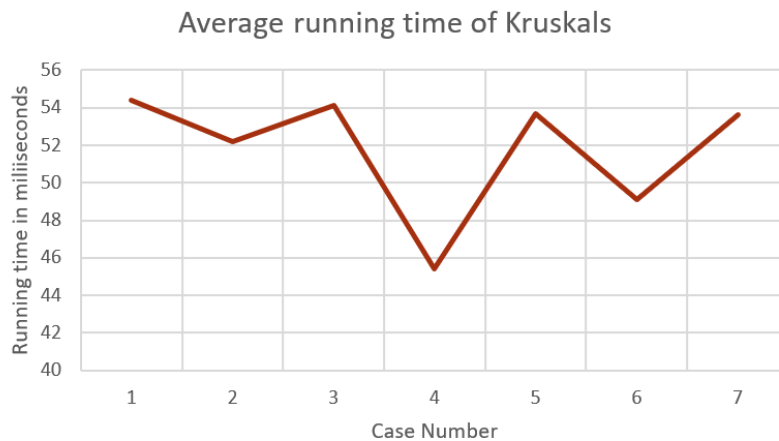


Figure 5: Average run time of Kruskal's algorithm

As shown in the above graph the average of all ten run times for Kruskal's algorithm finds that Kruskal runs slowly, so it is not more effective in computing a minimum spanning tree.

- The runtime average for each run's ten trials is shown in the plot above for Prim algorithm:

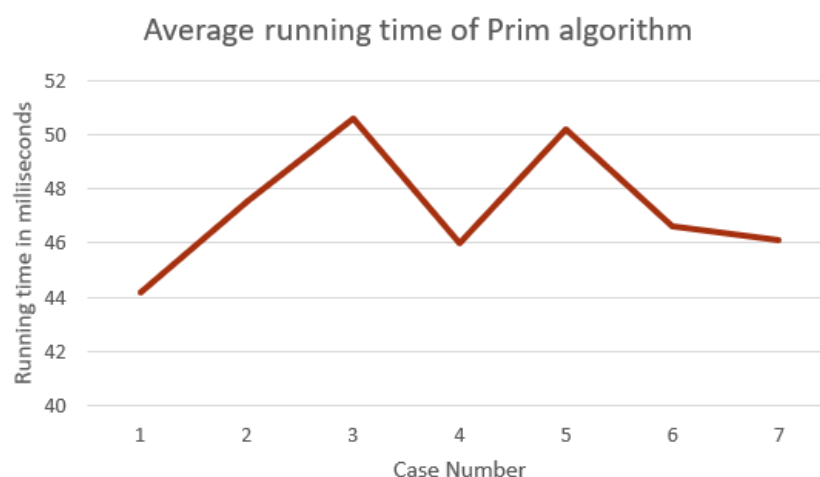


Figure 6: Average run time of Prim's algorithm

As shown in the above graph the average of all ten run times for the Prim algorithm finds that it runs very fast, so it is more effective than Kruskal's at finding the least spanning tree of the same graph.

- Screenshot of Requirement 2:

```
1: Read The Graph From File
2: Create a Random Graph

> From these choices, please enter your choice: 2
<< Test Cases >>

1: n = 1000 and m = 10000
2: n = 1000 and m = 15000
3: n = 1000 and m = 25000
4: n = 5000 and m = 15000
5: n = 5000 and m = 25000
6: n = 10000 and m = 15000
7: n = 10000 and m = 25000

< NOTE: n is the number of offices and m is the number of Lines >

> From these cases, please enter your choice: 1
Do you want the graph directed (Y/N) ? N
-----

Kruskal's Algorithm's run time is 58.0 ms.
-----

Min-heap based Prim Algorithm's run time is 0.0 ms.
-----

----- Thank You For Using Phone Network Design Program -----
PS C:\Users\LENOVO\Downloads\CPCS324-Projecr-Part1-main (1)\CPCS324-Projecr-Part1-main>
```

Figure 7: Output1 of Kruskal and Prim Algorithms for randomly generated graphic

```
1: Read The Graph From File
2: Create a Random Graph

> From these choices, please enter your choice: 2
<< Test Cases >>

1: n = 1000 and m = 10000
2: n = 1000 and m = 15000
3: n = 1000 and m = 25000
4: n = 5000 and m = 15000
5: n = 5000 and m = 25000
6: n = 10000 and m = 15000
7: n = 10000 and m = 25000

< NOTE: n is the number of offices and m is the number of Lines >

> From these cases, please enter your choice: 3
Do you want the graph directed (Y/N) ? n
-----

Kruskal's Algorithm's run time is 70.0 ms.
-----

Min-heap based Prim Algorithm's run time is 0.0 ms.
-----

----- Thank You For Using Phone Network Design Program -----
PS C:\Users\LENOVO\Downloads\CPCS324-Projecr-Part1-main (1)\CPCS324-Projecr-Part1-main>
```

Figure 8: Output1 of Kruskal and Prim Algorithms for randomly generated graphic



5. Difficulties Faced During the Phase Design

1. Limited knowledge of Using advanced functions of GitHub, such as Sharing and updating the project through GitHub.
2. Implementing algorithms by specific data structures that we do not commonly use it.
3. Difficulty indexing the vertices since the labels are string data types.
4. Difficulty Correct binding between the methods of classes that have a relation between them.
5. Validate the results and ensure the accuracy of the empirical analysis for Kruskal and Prim's Algorithms.

6. Conclusion

In conclusion, the empirical analysis project for Kruskal and Prim algorithms was conducted to compare the performance of these two algorithms in terms of runtime. The experiments were carried out on various sizes of randomly generated graphs and input graph from the txt file, and the results showed that Prim's algorithm performs better than Kruskal's algorithm in terms of both runtimes.

Specifically, the runtime of Prim's algorithm was found to be significantly lower than Kruskal's algorithm, especially for larger graph sizes.

Overall, the results of this empirical analysis project suggest that Prim's algorithm is a better choice than Kruskal's algorithm for solving minimum spanning tree problems, particularly for large graphs. However, it should be noted that the choice of algorithm may also depend on other factors such as the specific problem requirements and constraints.



7. References

(1) Levitin, A. (2011). Introduction to the Design and Analysis of Algorithms. Pearson Higher Ed.

8. Appendix

- GitHub Link:

<https://github.com/Razan-Alamri/CPCS324-Projecr-Part1>